Contrôle final R2.04a

Durée 2h

Seul document autorisé : 1 feuille A4 manuscrite

Machine à calculer (ou équipement similaire) interdite

Le sujet contient 3 exercices indépendants
Le barème mentionné est à titre indicatif

ATTENTION : une partie des réponses aux exercices sont à porter sur la feuille annexe jointe au sujet et à joindre à votre copie.

(pensez à **noter sur la feuille annexe votre nom et prénom**)

Exercice I : Adressage indirect *(6 points)*

Question I.1

Soit l'instruction **`MOVE.L 6(%A1),(%A1)+`**

Comment fonctionne l'adressage indirect post-incrémenté ?

Comment se décompose l'exécution de cette instruction ?

Question I.2

Soit l'extrait de programme suivant.
Sa première instruction (ligne 1) est stockée en mémoire à partir de l'adresse **%PC=0x0700**.

```
1        MOVE.W #10,%D1
2        MOVEA.L #0x00002025,%A1
3        MOVE.B (%A1),%D0
4        LEA.L 3(%A1),%A2
5        CMP.B 2(%A2),%D0
6        SUBA.L #8,%A2
7        MOVE.B -(%A1),%D0
8        MOVE.L (%A2,%D1),%D3
```

a) Indiquer dans le 1er tableau donné en dernière page du sujet (à joindre avec la copie), l'**équivalent algorithmique de chaque instruction**, pour les lignes 2 à 8.

b) Indiquer dans le 2nd tableau donné en dernière page du sujet (à joindre avec la copie) les valeurs successives des registres A1 et A2 **après** exécution de chaque instruction, ainsi que la valeur de l'adresse effective source quand c'est possible, de la ligne 2 à la ligne 8.

<u>Exercice II</u> : Appel de sous-programme *(2,5 points)*

Il s'agit de réaliser un programme calculant la somme de certains éléments d'un tableau d'entiers (TAB_VALEURS) en fonction d'une série d'indices (SEQUENCE).

```
/* Variables globales */
entier.w resultat;
entier.l SEQUENCE[8] = { 57, 32, 99, 6, 76, 20, 27, 43 } ;
entier.w TAB_VALEURS[100] = { /* 100 valeurs aléatoires entières */ };
```

```
1 entier.w FCT_somme(adresse.l adr_tableau, adresse.l adr_indices, entier.l nb_indices) ;
2
3  void main() {
4    // @ désigne l'adresse de la variable indiquée par son nom
5    resultat = FCT_somme(@TAB_VALEURS, @SEQUENCE, 8);
6  }
```

Le programme pseudo-Java ci-dessus est composé d'un programme principal (**`main`**) et d'un sous-programme nommé **`FCT_somme`** déclaré à la ligne 1(code à faire en exercice III). Donner les instructions en assembleur 68000 traduisant le code de la **ligne 5** et correspondant à un appel de sous-programme.

NB. on ne s'occupera pas de la déclaration des variables ni du programme principal (**`main`**).

<u>Exercice III</u> : Programmation de sous-programmes (11,5 points)

Question III.1 : sous-programme No 1 (3,5 pt)

```
1 entier.w FCT_donner_valeur(entier.l index, adresse.l adr_tableau)
2
3   A0.l = 2 * index + adr_tableau; // astuce : il est aussi possible d'utiliser
4                                   // l'instruction ADDA pour des calculs
5   D0.w = MEM[A0.l].w;             // arithmétiques pour entier non signés
6
7   return D0.w;
8 }
```

a) Réaliser le schéma de pile (dernière feuille du sujet) correspondant à son état lorsque le processeur exécute l'instruction en ligne 3 du sous-programme (aucune variable locale ni sauvegarde de contexte), en indiquant la <u>valeur de déplacement</u> par rapport au sommet de pile pour accéder aux différents arguments (**`adr_tableau, index`**).

b) Donner l'équivalent de ce sous-programme en assembleur 68000.

c) En ligne 3, pour quelle raison faut-il calculer **`2 * index`** pour lire la valeur d'un élément du tableau ? Expliquer.

Question III.2 : sous-programme No 2 (8 pt)

```
1 entier.w FCT_somme(adresse.l adr_tableau, adresse.l adr_indices, entier.l nb_indices){
2    entier.w somme = 0;  // variable locale
3
4    A1.l = adr_indices;
5    D1.l = 0;
6    while(D1.l < nb_indices) {
7       D2.l = MEM[4 * D1.l + A1.L]
8       somme = somme + FCT_donner_valeur(D2.l, adr_tableau) ;
9      D1.l = D1.l + 1;
10   }
11
12   return somme; // retourne la somme des éléments
13 }
```

Ce sous-programme s'appuie sur l'appel du sous-programme de la question III.1

a) Réaliser le schéma de pile (dernière feuille du sujet) correspondant à l'appel de ce sous-programme lors que le processeur exécute l'instruction ligne 4.

Le schéma doit inclure la sauvegarde du contexte et la variable locale **`somme`**.

Indiquer également sur le schéma la valeur de déplacement par rapport au sommet de pile pour accéder aux arguments (**`adr_tableau, adr_indices`**, et **`nb_indices`**).

b) Donner l'équivalent de ce sous-programme en assembleur 68000. Mettre un commentaire pour expliquer le branchement.

Annexe 1 : Instructions de branchement

$b_{11}..b_8$	B••	appellation	interprétation (dest) ? (source)	contexte d'utilisation	test sur les indicateurs
0111	BEQ	EQual	égal	quelconque	Z = 1
0110	BNE	Not Equal	différent	quelconque	Z = 0
0100	BCC	Carry Clear	≥ ; non dépassement	base deux	C = 0
0101	BCS	Carry Set	< ; dépassement	base deux	C = 1
0010	BHI	HIgh	>	base deux	C + Z = 0
0011	BLS	Less or Same	≤	base deux	C + Z = 1
1100	BGE	Greater or Equal	≥	complément à 2	N ⊗ V = 1
1101	BLT	Less Than	<	complément à 2	N ⊕ V = 1
1110	BGT	Greater Than	>	complément à 2	Z + (N ⊕ V) = 0
1111	BLE	Less or Equal	≤	complément à 2	Z + (N ⊕ V) = 1
1011	BMI	MInus	négatif	cpt à 2 ; ± ||	N = 1
1010	BPL	PLus	positif	cpt à 2 ; ± ||	N = 0
1000	BVC	oVerflow Clear	pas de débordement	complément à 2	V = 0
1001	BVS	oVerflow Set	débordement	complément à 2	V = 1
0000	BRA	True	vrai	quelconque	1
0001	*F*	False	faux	-	0

Annexe 2 : Instructions 68000

Récapitulatif partiel du jeu d'instructions

Mnémonique	Description	Fonctionnement	X	N	Z	V	C
ADD	Addition binaire	(Dest) + (Sour) → Dest	*	*	*	*	*
ADDI	Addition immédiate	(Dest) + don. imm. → Dest	*	*	*	*	*
ADDA	Addition adresses	(Dest) ET (Sour) → Dest	—	—	—	—	—
AND	ET logique	(Dest) ET (Sour) → Dest	—	*	*	0	0
ANDI	ET immédiat	(Dest) ET don. imm. → Dest	—	*	*	0	0
B..	Branchement conditionnel	si condition alors PC+d → PC	—	—	—	—	—
BRA	Branchement inconditionnel	PC+d → PC	—	—	—	—	—
BSET	Test de bit et mise à un	non détaillé ici	—	—	*	—	—
CLR	Mise à zéro de l'opérande	0 → Dest	—	0	1	0	0
CMP	Comparaison	(Dest) - (Sour) Dest est un registre de donnée Dn	—	*	*	*	*
CMPI	Comparaison immédiate	(Dest) - données immédiates	—	*	*	*	*
DIVS	Division signée	(Dest) / (Sour) → Dest	—	*	*	*	0
DIVU	Division non signée	(Dest) / (Sour) → Dest	—	*	*	*	0
EOR	OU exclusif logique	(Dest) ⊕ (Sour) → Dest	—	*	*	0	0
EORI	OU exclusif immédiat	(Dest) ⊕ don. imm. → Dest	—	*	*	0	0
EXG	Echange de registres	Rx ↔ Ry	—	—	—	—	—
EXT	Extension de signe	(Dest) signe étendu → Dest	—	*	*	0	0
JSR	Saut à un sous-pg	PC → -(SP) ; Dest → PC	—	—	—	—	—
LEA	Chargement adresse effective	@Sour → An	—	—	—	—	—
LSL, LSR	Décalage logique gauche/droite	(Dest) décalée <compte> → Dest	*	*	*	0	*
MOVE	Transfert de données	(Sour) → Dest	—	*	*	0	0
MOVEA	Transfert vers An	(Sour) → An	—	—	—	—	—
MOVEQ	Transfert rapide	Don. imm. → Dest	—	*	*	0	0
MULS	Multiplication signée	(Dest) * (Sour) → Dest	—	*	*	0	0
MULU	Multiplication non signée	(Dest) * (Sour) → Dest	—	*	*	0	0
NEG	Complément à 2	0 - (Dest) → Dest	*	*	*	*	*
NOP	Non opération		—	—	—	—	—
NOT	Complément logique	~(Dest) → Dest	—	*	*	0	0
OR	OU logique inclusif	(Dest) OU (Sour) → Dest	—	*	*	0	0
PEA	Empilement adresse effective	Dest → -(SP)	—	—	—	—	—
RTE	Retour d'exception	(SP)+ → SR ; (SP)+ → PC	*	*	*	*	*
RTS	Retour de sous-programme	(SP)+ → PC	—	—	—	—	—
SUB	Soustraction	(Dest) - (Sour) → Dest	*	*	*	*	*
SUBA	Soustraction adresse	(Dest) - (Sour) → Dest	—	—	—	—	—
SUBI	Soustraction immédiate	(Dest) - don. imm. → Dest	*	*	*	*	*
TST	Test d'un opérande	(Dest) testé. → cc	—	*	*	0	0

NOM : _____ **Prénom :** _____
(feuille à joindre à la copie)

<u>Exercice I</u> :

Question I.2.a Equivalent algorithmique des instructions.

`MOVEA.L #0x00002025,%A1`	
`MOVE.B (%A1),%D0`	
`LEA.L 3(%A1),%A2`	
`CMP.B 2(%A2),%D0`	
`SUBA.L #8,%A2`	
`MOVE.B -(%A1),%D0`	
`MOVE.L (%A2,%D1),%D3`	

Question I.2.b Valeurs successives de A1 et A2, de la valeur d'adresse effective source.

	A1 (.L)	A2 (.L)	Adresse effective pour la source
`MOVEA.L #0x00002025,%A1`		XXXXXXXXXXXX	(difficile)
`MOVE.B (%A1),%D0`		XXXXXXXXXXXX	
`LEA.L 3(%A1),%A2`	XXXXXXXXXXXXX		XXXXXXXXXXXXX
`CMP.B 2(%A2),%D0`	XXXXXXXXXXXXX		
`SUBA.L #8,%A2`	XXXXXXXXXXXXX		XXXXXXXXXXXXX
`MOVE.B -(%A1),%D0`		XXXXXXXXXXXX	
`MOVE.L (%A2,%D1),%D3`			

(suite au verso)

Exercice III :

Question III.1	Question III.2